Workflow Templates — Plan napraw

> *Wygenerowano: 2026-03-30 | Źródło: analiza v1 + v2 + porównanie*

> *Katalog szablonów:* `/workspace/sembot-laravel/workflow-templates/`

> *Dokumentacja:* `/workspace/sembot-laravel/.doc/workflow/`

STATUS WYKONANIA

Etap	Zadanie	Status
1	GDPR — usunięcie danych wrażliwych	⬜ TODO
2	Krytyczne przebudowy (#4, #14)	⬜ TODO
3	Trigger bug — `days_of_month`	⬜ TODO
4	camelCase → snake_case w Google getterach	⬜ TODO
5	`parent_id: null` — naprawy połączeń	⬜ TODO
6	Złe nazwy pól w task description	⬜ TODO
7	Guard na dzielenie przez zero	⬜ TODO
8	Iterator przed AI agentem	⬜ TODO
9	`agent_id: null` — dokumentacja i placeholder	⬜ TODO
10	Weryfikacja API contracts (nazwy pól DTO)	⬜ TODO
11	Ulepszenia drugorzędne	⬜ TODO
12	Weryfikacja końcowa	⬜ TODO

ETAP 1 — GDPR: Usunięcie danych wrażliwych

> ***Priorytet: BLOKUJĄCY** — szablony nie mogą być udostępnione dopóki zawierają dane pracowników*

TASK 1.1 — `budget_deviation.json`

Problem: Plik zawiera prawdziwe dane użytkownika Sembota — nie jest szablonem, lecz realnym workflow.

Do usunięcia/zastąpienia:

- `creator.email: "lukas.lochocki@sembot.io"` → `""`
- `creator.name` (jeśli istnieje) → `""`
- `connection_id: 652` → `null` lub `""`
- `project_id: 909` → `null` lub `""`
- Każde wystąpienie `customer_id: 8725857420` wewnątrz nodes → `null` lub `""`
- `status: "active"` → `"inactive"`

Weryfikacja po zmianie:

- ⬜ `grep -r "lochocki"` w pliku zwraca zero wyników
- ⬜ `grep -r "8725857420"` zwraca zero wyników
- ⬜ `status` = `"inactive"`

TASK 1.2 — `conversion_comparison.json`

Problem: Analogicznie — realny workflow użytkownika z emailem pracownika.

Do usunięcia/zastąpienia:

- `creator.email: "michal.strzebala@sembot.io"` → `""`
- `connection_id` (hardcoded) → `null` lub `""`
- `project_id` (hardcoded) → `null` lub `""`
- `customer_id` (hardcoded) → `null` lub `""`
- `status: "active"` → `"inactive"`
- `timezone: "Africa/Abidjan"` → `"Europe/Warsaw"` (był timezone przypadkowego użytkownika)

Weryfikacja po zmianie:

- ⬜ `grep -r "strzebala"` zwraca zero wyników
- ⬜ `status` = `"inactive"`
- ⬜ `timezone` = `"Europe/Warsaw"` lub sensowna domyślna

ETAP 2 — Krytyczne przebudowy

> *Szablony wymagające przepisania od zera — nie wystarczy patch*

TASK 2.1 — `budget_protector.json` — pełna przebudowa

Problem: Aktywny domyślnie, odpala co godzinę, getter odłączony od triggera, `globals: []` (tablica zamiast obiektu), hardcoded próg `cost > 100`.

Wzorzec do naśladowania: `ms_budget_protector.json` (działa poprawnie)

Wymagania nowej wersji:

- ⬜ `status: "inactive"` — nigdy nie aktywować automatycznie przy imporcie
- ⬜ Nowy format trigger: `type: "schedule"` zamiast `trigger_type: "cron"`
- ⬜ `frequency: "daily"` z interwałem — nie co godzinę (flood tasków). Rozważyć co 4h (`interval: 4`, `frequency: "hourly"`) lub `frequency: "daily"` rano
- ⬜ `globals: { budget_threshold_pct: 90, min_spend: 10 }` — konfigurowalne progi zamiast hardcoded `cost > 100`
- ⬜ Getter: poprawny `parent_id` wskazujący na trigger node
- ⬜ Filtr: `percentageSpend > globals.budget_threshold_pct` zamiast `cost > 100`
- ⬜ Iterator wskazuje na filtr (nie getter) — `sourceId` = ID filtra
- ⬜ Task description: zawiera kampanię, aktualny koszt, budżet, procent wykorzystania
- ⬜ `usedBy: []`, `tags: []`, `globals` jako obiekt `{}`

Struktura węzłów (wzorzec):

```
[1] trigger/schedule → [2] getter/get_google_campaigns → [3] filter → [4] iterator(sourceId:3) → [5] action/create
```

TASK 2.2 — `pmax_analysis.json` — naprawa połączeń węzłów

Problem: 4 węzły z `parent_id: null` — cały workflow rozłączony, nic się nie wykona.

Podejście: Najpierw przeczytać plik i zrozumieć zamierzoną strukturę, potem naprawić `parent_id`.

Oczekiwana struktura logiczna:

```
[1] trigger → [2] getter_campaigns → [3] getter_assets → [4] ai_agent → [5] create_task
```

lub (jeśli dwa równoległe gettery):

```
[1] trigger → [2] getter_campaigns (parent: 1)
[1] trigger → [3] getter_assets (parent: 1)
[4] ai_agent (parent: ???) — problem: brak merge node
[5] create_task (parent: 4)
```

Subtaski:

- ⬜ Przeczytać `pmax_analysis.json` i zmapować aktualne `id` → `parent_id`
- ⬜ Jeśli dwa równoległe gettery → rozważyć czy silnik workflow obsługuje merge; jeśli nie — uczynić sekwencyjnym
- ⬜ Ustawić `parent_id` dla każdego node'a na poprawne ID poprzednika
- ⬜ Sprawdzić czy `connection_id`, `project_id` są puste (template)
- ⬜ `status: "inactive"`
- ⬜ Zweryfikować czy getter `get_google_pmax_campaigns` istnieje w systemie (sprawdzić w kodzie lub dokumentacji)

TASK 2.3 — `visibility_drop_alert.json` — przebudowa logiki

Problem: Szablon wykrywa "złe pozycje" zamiast "spadki pozycji". Nazwa "drop alert" sugeruje porównanie w czasie, ale nie ma dual-getter ani historycznych danych.

Nowa architektura (drop detection):

```
[1] trigger/weekly → [2] getter/current_week (7 dni) → [3] getter/previous_week (7 dni) → [4] filter/percentageCha
```

Subtaski:

- ⬜ Zmienić na nowy format
- ⬜ Dodać dual-getter: bieżący tydzień vs poprzedni tydzień
- ⬜ Filtr: `percentageChange(current_rank, previous_rank) > globals.drop_threshold` (np. 20%)
- ⬜ Dodać `globals: { drop_threshold_pct: 20, min_search_volume: 100 }` — ignoruj frazy z minimalnym wolumenem
- ⬜ Naprawić `parent_id` — getter musi być połączony z triggerem
- ⬜ Zweryfikować rzeczywiste nazwy pól w SEO getter DTO: `rank_absolute` czy `rankAbsolute`?
- ⬜ `status: "inactive"`

TASK 2.4 — `inventory_aware_alert.json` — weryfikacja i naprawa

Problem: `"out_of_stock"` (underscore) vs GMC API `"out of stock"` (spacja) — prawdopodobnie zero dopasowań. Trzeba zweryfikować rzeczywisty format.

Subtaski:

- ⬜ Sprawdzić w kodzie PHP domenę GMC (`Domain/GoogleMerchantCenter/`) jaki format zwraca pole `availability` w DTO
- ⬜ Wyszukać: `grep -r "availability" /workspace/sembot-laravel/Domain/GoogleMerchantCenter/ --include="*.php"`
- ⬜ Jeśli format to `"out of stock"` (spacja) → zmienić w filtrze szablonu
- ⬜ Jeśli format to `"out_of_stock"` (underscore) → szablon jest OK w tym zakresie
- ⬜ Naprawić `parent_id: null` na getterze
- ⬜ Puste `connection_id: ""`, `project_id: ""`
- ⬜ `status: "inactive"`
- ⬜ Zmienić na nowy format trigger

TASK 2.5 — `ms_seasonal_ad_copy_check.json` — zmiana gettera

Problem: Getter pobiera kampanie zamiast treści reklam → AI nie widzi żadnego ad copy do analizy.

Subtaski:

- ⬜ Zidentyfikować getter dla treści reklam w Microsoft Ads — szukać w `Domain/MicrosoftAdvertising/` lub dokumentacji getterów
- ⬜ Sprawdzić: czy istnieje `get_microsoft_advertising_ads` lub podobny?
- ⬜ Zastąpić getter kampanii getterem reklam
- ⬜ Zaktualizować pola w prompcie AI i task description na pola zwracane przez getter reklam
- ⬜ Wzorzec: `seasonal_ad_copy_check.json` (Google wersja) — adaptować na MS

ETAP 3 — Trigger bug: `days_of_month: "first_day"`

> ***Problem systemowy:*** *`frequency: daily` lub `frequency: weekly` połączone z `days_of_month: "first_day"` powoduje że trigger odpala się TYLKO 1. dnia miesiąca.*

> ***Fix:*** *Dla szablonów z `frequency: daily` lub `frequency: weekly` — usunąć pole `days_of_month` całkowicie.*

> ***Zostawić:*** *Tylko gdy `frequency: monthly` (tam `days_of_month` ma sens).*

TASK 3.1 — `budget_deviation.json`

- ⬜ Sprawdzić `frequency` wartość
- ⬜ Usunąć `days_of_month: "first_day"` (frequency jest `daily` → bug)
- ⬜ Upewnić się że trigger odpala codziennie

TASK 3.2 — `conversion_comparison.json`

- ⬜ Usunąć `days_of_month` — trigger `hourly` + `first_day` = odpala co godzinę tylko 1. dnia miesiąca

TASK 3.3 — `price_competitiveness_alert.json`

- ⬜ `frequency: daily` + `days_of_month: "first_day"` → usunąć `days_of_month`
- ⬜ Opis szablonu mówi "dzienny alert" — trigger musi być faktycznie dzienny

TASK 3.4 — `daily_ppc_brief.json`

- ⬜ `frequency: daily` + `days_of_month: "first_day"` → usunąć `days_of_month`
- ⬜ "Daily PPC Brief" odpalany miesięcznie to nonsens

TASK 3.5 — `ms_budget_protector.json`

- ⬜ Sprawdzić czy ma `days_of_month: "first_day"`
- ⬜ Jeśli tak i `frequency != monthly` → usunąć

TASK 3.6 — `ms_budget_deviation.json`

- ⬜ Analogicznie jak 3.5

TASK 3.7 — Skan wszystkich pozostałych szablonów

- ⬜ Wykonać: `grep -l "first_day" /workspace/sembot-laravel/workflow-templates/*.json`
- ⬜ Dla każdego znalezionego — sprawdzić `frequency` i zdecydować czy `days_of_month` ma sens
- ⬜ Szablon miesięczny z `days_of_month: "first_day"` jest POPRAWNY — nie ruszać
- ⬜ Szablon dzienny/tygodniowy z `days_of_month: "first_day"` — usunąć pole

ETAP 4 — camelCase → snake_case w parametrach Google getterów

> ***Problem systemowy:*** *Stare szablony używają* `startDate` / `endDate` / `sortField` / `sortOrder` *zamiast* `start_date` / `end_date` / `sort_field` / `sort_order`*. Powoduje to że getter ignoruje parametry dat i sortowania.*

> ***Przed naprawą:*** *Zweryfikować rzeczywiste nazwy parametrów w kodzie gettera.*

> *Sprawdzić:* `grep -r "start_date\|startDate" /workspace/sembot-laravel/Domain/GoogleAds/ --include="*.php" -l`

TASK 4.1 — Weryfikacja rzeczywistych nazw parametrów

- ⬜ Znaleźć w kodzie jak getter przyjmuje parametry (DTO lub handler)
- ⬜ Sprawdzić dla: `get_google_ads_metrics`, `get_google_search_ads`, `get_google_keywords`
- ⬜ Zapisać: używa `start_date` czy `startDate`? Jest jedno źródło prawdy w kodzie PHP.
- ⬜ Jeśli kod używa `start_date` → wszystkie szablony z camelCase są broken
- ⬜ Jeśli kod akceptuje oba → problem jest mniejszy, ale ujednolicić na snake_case

TASK 4.2 — Szablony Google do poprawki (po weryfikacji)

Dla każdego z poniższych — zmienić `startDate` → `start_date`, `endDate` → `end_date`, `sortField` → `sort_field`, `sortOrder` → `sort_order`:

- ⬜ `non_converting_keywords.json` — getter node params
- ⬜ `trending_search_terms.json` — getter node params
- ⬜ `ai_weekly_report.json` — getter node params
- ⬜ `campaign_intelligence_scoring.json` — getter node params
- ⬜ `seasonal_ad_copy_check.json` — getter node params
- ⬜ `daily_ppc_brief.json` — getter node params
- ⬜ `weekly_opportunity_scout.json` — getter node params
- ⬜ `search_terms_clustering.json` — getter node params
- ⬜ `keyword_quality_audit.json` — getter node params
- ⬜ `master_account_audit.json` — oba gettery (Google + ?)
- ⬜ `competitor_gap_analysis.json` — getter node params
- ⬜ `low_ctr_detective.json` — getter node params
- ⬜ `ad_copy_quality_scorer.json` — getter node params
- ⬜ `ad_copy_message_audit.json` — getter node params

ETAP 5 — Naprawy `parent_id: null`

> ***Problem:*** *Węzły z `parent_id: null` nie są podłączone do łańcucha wykonania — nigdy się nie uruchomią.*

TASK 5.1 — `budget_protector.json`

- ⬜ Getter node: ustawić `parent_id` = ID node'a triggera (zazwyczaj `1`)

TASK 5.2 — `visibility_drop_alert.json`

- ⬜ Getter node: `parent_id` = ID triggera

TASK 5.3 — `inventory_aware_alert.json`

- ⬜ Getter node: `parent_id` = ID triggera

TASK 5.4 — `non_converting_keywords.json`

- ⬜ Getter node: `parent_id` = ID triggera

TASK 5.5 — `trending_search_terms.json`

- ⬜ Getter node: `parent_id` = ID triggera

TASK 5.6 — `pmax_analysis.json`

- ⬜ Patrz TASK 2.2 — pełna analiza i naprawa wszystkich 4 odłączonych węzłów

TASK 5.7 — Skan wszystkich szablonów

- ⬜ `grep -n "parent_id.null" /workspace/sembot-laravel/workflow-templates/.json`
- ⬜ Dla każdego trafienia: ocenić czy `null` jest celowy (np. trigger zawsze ma `parent_id: null`) czy błędny (getter, filtr, action powinny mieć `parent_id`)
- ⬜ Trigger node (type: schedule/cron) → `parent_id: null` jest POPRAWNY
- ⬜ Wszystkie inne typy węzłów → `parent_id: null` jest BŁĘDEM

ETAP 6 — Złe nazwy pól w task description

> *Pola `{node_id.field_name}` w task description odwołują się do nieistniejących pól → wyświetlają `null` lub pusty string.*

TASK 6.1 — Weryfikacja nazw pól w DTO

Przed naprawą szablonów — zweryfikować rzeczywiste nazwy w kodzie:

- ⬜ **Google keyword field:** `grep -r "keyword_text\|keywordText" /workspace/sembot-laravel/Domain/GoogleAds/ --include="*.php"` — sprawdzić jak pole nazywa się w DTO keywords gettera
- ⬜ **Google search term field:** `grep -r "search_term\|searchTerm" /workspace/sembot-laravel/Domain/GoogleAds/ --include="*.php"` — weryfikacja
- ⬜ **MS search query field:** `grep -r "searchQuery\|search_query" /workspace/sembot-laravel/Domain/MicrosoftAdvertising/ --include="*.php"` — weryfikacja
- ⬜ **RSA headline fields:** `grep -r "headline" /workspace/sembot-laravel/Domain/GoogleAds/ --include="*.php"` — jak nazywają się pola nagłówków reklam RSA

TASK 6.2 — Naprawy w szablonach Google

- ⬜ `non_converting_keywords.json`: `{4.title}` → `{4.keyword_text}` (po weryfikacji nazwy)
- ⬜ `trending_search_terms.json`: `{4.title}` → `{4.search_term}` (po weryfikacji)
- ⬜ `weekly_opportunity_scout.json`: `{2.title}` → `{2.search_term}` (po weryfikacji, uwzględniając że może być `{3.search_term}` jeśli jest filtr jako node 2)
- ⬜ `search_terms_clustering.json`: `{2.title}` → `{2.search_term}` (j.w.)
- ⬜ `master_account_audit.json`: `{3.search_term}` — zweryfikować czy to właściwy node ID
- ⬜ `competitor_gap_analysis.json`: pola `isMyResult` → `is_my_result` (jeśli snake_case)

TASK 6.3 — Naprawy w szablonach RSA

- ⬜ `seasonal_ad_copy_check.json`: `headline1`/`headline2`/`headline3` — po weryfikacji zastąpić prawidłowymi nazwami lub strukturą tablicową
- ⬜ `ad_copy_quality_scorer.json`: j.w.
- ⬜ `ad_copy_message_audit.json`: j.w.

TASK 6.4 — Niespójność MS Ads search query field

- ⬜ `ms_trending_search_terms.json`: `{2.searchQuery}` — zweryfikować czy to właściwa nazwa
- ⬜ `ms_ads_winners.json`: `{4.searchTerm}` — zmienić na `{4.searchQuery}` (per wzorzec `microsoft_ads_waste_finder.json` który używa `searchQuery`)
- ⬜ Ustalić canonical field name dla MS search queries i ujednolicić we wszystkich MS szablonach

---

ETAP 7 — Guard na dzielenie przez zero

> ***Problem:*** `CPA = cost / conversions` *bez sprawdzenia* `conversions > 0` *→ błąd runtime lub fałszywy alarm.*

TASK 7.1 — `cpa_campaign_alert.json` / `budget_alert.json`

- ⬜ W filtrze dodać warunek: `conversions > 0` jako dodatkowy filtr przed obliczeniem CPA
- ⬜ Alternatywnie: filtr na wyrażenie z zabezpieczeniem (jeśli silnik to obsługuje)

TASK 7.2 — `roas_drop_alert.json`

- ⬜ Dodać `conversions_value > 0` lub `revenue > 0` w filtrze (przed obliczeniem ROAS)

TASK 7.3 — `conversion_comparison.json`

- ⬜ Dodać guard `conversions > 0` dla obu okresów (bieżący i poprzedni)

TASK 7.4 — `ms_roas_drop_alert.json`

- ⬜ Dodać `revenue > 0` w filtrze (MS Ads używa `revenue` zamiast `conversions_value`)

TASK 7.5 — `budget_deviation.json` i `ms_budget_deviation.json`

- ⬜ Problem: `divide(subtract(budget, cost), subtract(getDaysInMonth(), getDayOfMonth()))` → dzielenie przez zero na ostatni dzień miesiąca (`getDaysInMonth() - getDayOfMonth() = 0`)
- ⬜ Dodać warunek: nie wykonuj workflow w ostatnim dniu miesiąca, lub użyj `max(1, subtract(getDaysInMonth(), getDayOfMonth()))` jako mianownik

---

ETAP 8 — Iterator przed AI agentem

> ***Problem:*** *AI agent otrzymuje tablicę danych bez iteratora — przetwarza tylko 1. element lub otrzymuje serializowaną tablicę jako string.*

> ***Wzorzec poprawny:*** `filter → iterator(sourceId: filter_id) → ai_agent`

TASK 8.1 — `ai_weekly_report.json`

- ⬜ Dodać `iterator` node między getterem/filtrem a AI agentem
- ⬜ `iterator.sourceId` = ID filtra (lub gettera jeśli brak filtra)
- ⬜ Upewnić się że w prompcie AI używane są `{node_id.field}` z właściwego węzła

TASK 8.2 — `campaign_intelligence_scoring.json`

- ⬜ Analogicznie jak 8.1

TASK 8.3 — `daily_ppc_brief.json`

- ⬜ Dodać iterator. Uwaga: brief dzienny może chcieć summary wszystkich kampanii — rozważyć czy iterator (jedna kampania per AI call) czy zbiorczy prompt (wszystkie kampanie naraz) jest właściwym podejściem. Jeśli zbiorczy → zostawić bez iteratora ale upewnić się że prompt wie że dostaje tablicę.

TASK 8.4 — `ms_ai_weekly_report.json`

- ⬜ Analogicznie jak 8.1

TASK 8.5 — `ms_campaign_intelligence_scoring.json`

- ⬜ Analogicznie jak 8.2

TASK 8.6 — Weryfikacja szablonów AI które mają iterator

- ⬜ `seasonal_ad_copy_check.json` — sprawdzić czy iterator jest przed AI (powinien być ✅)
- ⬜ `microsoft_ads_waste_finder.json` — sprawdzić czy iterator jest przed AI (powinien być ✅)
- ⬜ `weekly_opportunity_scout.json` — sprawdzić pozycję iteratora
- ⬜ `search_terms_clustering.json` — sprawdzić. Uwaga: clustering może działać na wszystkich naraz (AI dostaje pełną listę) — to może być celowe

---

ETAP 9 — `agent_id: null`

> ***Problem:*** *AI agent bez przypisanego `agent_id` — silnik rzuca błąd lub pomija węzeł.*

> ***Decyzja do podjęcia:*** *Czy przypisać konkretny domyślny `agent_id`, czy zostawić null z dokumentacją?*

TASK 9.1 — Ustalenie polityki

- ⬜ Sprawdzić w kodzie jak silnik workflow obsługuje `agent_id: null` — błąd, skip, czy default?
- ⬜ Sprawdzić: `grep -r "agent_id" /workspace/sembot-laravel/Domain/Task/ --include="*.php"`
- ⬜ Jeśli `null` powoduje błąd → szablony z AI są broken dopóki user nie skonfiguruje
- ⬜ Ustalić: czy jest "domyślny agent" który można wpisać? Czy trzeba null zastąpić placeholder ID?

TASK 9.2 — Dokumentacja w szablonach

- ⬜ Dodać w `README.md` szablonów sekcję: "Szablony AI wymagają konfiguracji — po imporcie ustaw agent_id w węzłach AI"
- ⬜ Opcjonalnie: dodać `_requires_configuration: ["agent_id"]` jako metadane do szablonów AI

TASK 9.3 — Lista szablonów AI wymagających `agent_id`

Wszystkie poniższe mają `agent_id: null`:

- `ai_weekly_report.json`
- `campaign_intelligence_scoring.json`
- `daily_ppc_brief.json`
- `weekly_opportunity_scout.json`
- `search_terms_clustering.json`
- `monthly_cross_platform_summary.json`
- `keyword_quality_audit.json`
- `master_account_audit.json`
- `competitor_gap_analysis.json`
- `low_ctr_detective.json`
- `ad_copy_quality_scorer.json`
- `ad_copy_message_audit.json`
- `seasonal_ad_copy_check.json`
- `ms_ai_weekly_report.json`
- `ms_campaign_intelligence_scoring.json`
- `ms_seasonal_ad_copy_check.json`

ETAP 10 — Weryfikacja API contracts (nazwy pól DTO)

> ***Cel:*** *Potwierdzić lub poprawić założenia z v2 dotyczące nazw pól — zanim zmienimy szablony.*

TASK 10.1 — Google Ads gettery

- ⬜ `get_google_keywords` lub podobny — jak nazywa się pole słowa kluczowego w DTO? `keyword_text`? `text`? `keyword`?
- ⬜ `get_google_search_terms` — jak nazywa się pole frazy wyszukiwania? `search_term`? `query`?
- ⬜ `get_google_ads_metrics` — czy istnieje pole `ctr`? W jakim formacie (decimal 0.032 czy procent 3.2)?
- ⬜ RSA reklamy — jak nazywają się pola nagłówków? `headline_1`? `headlines[]`? `headlinePart1`?
- ⬜ Jak nazywa się pole `conversions` vs `conversions_value` vs `all_conversions` w metrics DTO?

Metoda: `grep -r "class.DTO\|public readonly" /workspace/sembot-laravel/Domain/GoogleAds/ --include=".php" -A 20 | head -100`

TASK 10.2 — Microsoft Advertising gettery

- ⬜ `get_microsoft_advertising_search_terms` — `searchQuery`? `search_query`? `query`?
- ⬜ `get_microsoft_advertising_campaigns` — `spend`? `cost`? `costPerConversion`?
- ⬜ Jak nazywa się przychód? `revenue`? `conversions_value`?

TASK 10.3 — Google Merchant Center gettery

- ⬜ `get_google_merchant_center_products` — `availability` — jaki dokładnie format? `"in stock"`, `"out of stock"`, `"in_stock"`, `"out_of_stock"`?
- ⬜ Czy `benchmarkPriceMicros` to rzeczywista nazwa pola z API czy alias?
- ⬜ Format cen — mikros (1 PLN = 1000000) — potwierdzić

ETAP 11 — Ulepszenia drugorzędne

> *Nie blokują działania — poprawiają jakość, UX i bezpieczeństwo*

TASK 11.1 — Konwersja mikrosów w GMC szablonach

- ⬜ `price_competitiveness_alert.json`: W task description dodać konwersję `divide({node.priceMicros}, 1000000)` zamiast surowej wartości
- ⬜ `inventory_aware_alert.json`: Analogicznie jeśli zawiera ceny

TASK 11.2 — Ujednolicenie formatu CTR

- ⬜ Ustalić: czy getter Google zwraca CTR jako `0.032` (decimal) czy `3.2` (procent)?
- ⬜ `cpa_campaign_alert.json` i `low_ctr_detective.json`: zaktualizować progi i wyświetlanie
- ⬜ `low_ctr_detective.json`: `ctr_threshold: 2` — jeśli CTR jest decimal → zmienić na `0.02`

TASK 11.3 — Iterator `sourceId` w MS szablonach

- ⬜ `ms_cpa_campaign_alert.json`: iterator `sourceId` wskazuje na getter (node 2) zamiast filtr (node 3) — zmienić
- ⬜ `ms_roas_drop_alert.json`: analogicznie
- ⬜ `ms_zero_impression_campaigns.json`: analogicznie
- ⬜ `ms_budget_protector.json`: analogicznie
- ⬜ Wzorzec poprawny: `ms_non_converting_keywords.json` i `microsoft_ads_waste_finder.json` — iterator wskazuje na filtr

TASK 11.4 — Zbiorczy task zamiast task-per-entity

- ⬜ `non_converting_keywords.json`: rozważyć zmianę z "task per słowo" na "jeden task z listą słów" — mniej zaśmiecenia task listy
- ⬜ `trending_search_terms.json`: analogicznie — zamiast 20 tasków jeden task z tabelą TOP 20
- ⬜ `ad_copy_quality_scorer.json`: limit 500 tasków/miesiąc — dodać `globals: { limit: 50 }` z rozsądnym domyślnym
- ⬜ Uwaga: ta zmiana wpływa na architekturę (usuwa iterator) — wymaga świadomej decyzji produktowej

TASK 11.5 — `ms_ads_winners.json` — stary format + active

- ⬜ Zmienić na nowy format
- ⬜ `status: "inactive"`
- ⬜ `{4.searchTerm}` → `{4.searchQuery}` (po weryfikacji)

TASK 11.6 — `ms_conversion_comparison.json` — global `conversion_cost` nieużywany

- ⬜ Sprawdzić czy `conversion_cost` jest rzeczywiście zdefiniowany ale nieużywany
- ⬜ Jeśli tak: albo użyć go w filtrze albo usunąć z `globals`

TASK 11.7 — Szablony-duplikaty

- ⬜ `ad_copy_quality_scorer.json` i `ad_copy_message_audit.json` — mają identyczną architekturę, tylko prompt AI różni je. Rozważyć scalenie lub jawne oznaczenie że są intencjonalnie różne.

ETAP 12 — Weryfikacja końcowa

TASK 12.1 — Skan bezpieczeństwa (po wszystkich naprawach)

- ⬜ `grep -r "active" /workspace/sembot-laravel/workflow-templates/*.json | grep "status"` — żaden szablon nie powinien mieć `status: "active"`
- ⬜ `grep -r "@sembot.io\|@gmail\|connection_id.[0-9]\{3,\}\|project_id.[0-9]\{3,\}" /workspace/sembot-laravel/workflow-templates/*.json` — sprawdzenie czy nie zostały emaile lub hardcoded ID
- ⬜ `grep -r "parent_id.null" /workspace/sembot-laravel/workflow-templates/.json` — tylko trigger node powinien mieć null
- ⬜ `grep -r "globals.\[\]" /workspace/sembot-laravel/workflow-templates/.json` — `globals` zawsze jako `{}` nie `[]`

TASK 12.2 — Import testowy

- ⬜ Przetestować import każdego poprawionego szablonu przez UI lub API
- ⬜ Sprawdzić czy workflow renderuje poprawnie w edytorze (wszystkie węzły połączone)
- ⬜ Sprawdzić czy trigger ma sensowny schedule

TASK 12.3 — Aktualizacja dokumentacji

- ⬜ Zaktualizować `README.md` w katalogu `workflow-templates/`
- ⬜ Zaktualizować `PLAN.md` (jeśli istnieje) o listę naprawionych błędów
- ⬜ Opcjonalnie: wygenerować PDF v3 z wynikami po naprawach

Kolejność prac (sugerowana)

```
Etap 1 (GDPR)          → bezwzględnie pierwsze
Etap 2 (przebudowy)    → #budget_protector, #pmax_analysis, #visibility_drop_alert
Etap 10 (weryfikacja)  → zrobić PRZED etapami 4, 6 — żeby nie zgadywać nazw pól
Etap 3 (trigger bug)   → prosty, masowy fix
Etap 5 (parent_id)     → prosty, masowy fix
Etap 4 (camelCase)     → po etapie 10 (zweryfikowane nazwy)
Etap 6 (field names)   → po etapie 10
Etap 7 (guards /0)     → niezależny, można w dowolnym momencie
Etap 8 (iterator+AI)   → po etapie 2.2 (pmax) i 2.3 (visibility)
Etap 9 (agent_id)      → wymaga decyzji produktowej — na końcu
Etap 11 (ulepszenia)   → na końcu, nie blokuje
Etap 12 (weryfikacja)  → zawsze ostatnie
```

Metryki sukcesu

Po zakończeniu wszystkich napraw:

- ⬜ 0 szablonów z `status: "active"`
- ⬜ 0 szablonów z emailami pracowników lub hardcoded ID klientów
- ⬜ 0 szablonów z `parent_id: null` na węzłach innych niż trigger
- ⬜ 0 szablonów z `globals: []` (tablica) — wszystkie `globals: {}`
- ⬜ Trigger bug naprawiony we wszystkich szablonach z `daily/weekly + first_day`
- ⬜ Znane nazwy pól DTO dla wszystkich getterów (udokumentowane)
- ⬜ Każdy szablon AI ma dokumentację że wymaga konfiguracji `agent_id`

Ostatnia aktualizacja: 2026-03-30